Introduction

Large language models (LLMs) such as ChatGPT, Gemini and Claude are now used every day to write emails, essays, articles and explanations. Their texts are fluent and often hard to tell apart from human writing, and even harder to tell apart from each other. Identifying **which model wrote a text** has become an important problem in text mining. It supports AI-text detection in education and journalism, helps with authorship and copyright questions, and helps us understand how models differ in their behaviour.

Each language model develops its own **writing habits** during training: the connector words it prefers, how formal it is, whether it uses contractions or speaks to the reader directly, and how it structures a sentence. These habits act like a fingerprint. If they can be measured, a machine learning model can learn to recognise them.

This project focuses on a particularly difficult version of the problem: distinguishing between **five models from the same family**, Anthropic's Claude (claude-fable-5, claude-opus-4-6, claude-opus-4-8, claude-opus-5 and claude-sonnet-5). Because these models were built by the same company, their style differences are expected to be subtle.

The project has two research questions:

1. **Can the model that wrote a passage be identified from its text alone?**
2. **Which words give away each AI model?**

I used the *Name That LLM – Extra API Passages* dataset from Kaggle. It contains 4,998 single-paragraph texts, in which each of the five models answered the same 1,000 prompts. Because every model received the same prompts, the differences between the passages come from writing style rather than topic.

The project follows a standard data mining pipeline:

- **Data exploration:** analysing the class balance, text lengths, domains and the most common words of each model.
- **Text preprocessing:** checking for missing and duplicate values and cleaning the text.
- **Feature extraction:** converting the text into numbers with **TF-IDF** (Term Frequency–Inverse Document Frequency), using single words and two-word phrases.
- **Modelling:** training and comparing three classifiers commonly used for text, namely **Multinomial Naive Bayes**, **Logistic Regression** and a **Linear Support Vector Machine**, evaluated with accuracy, precision, recall, F1-score, confusion matrices, grouped cross-validation and hyperparameter tuning.
- **Interpretation:** using the Logistic Regression coefficients to find the words and phrases most typical of each model, and the confusion matrix to find which models are most often mixed up.

Since the five classes are balanced, a random guess would be correct only **20%** of the time. This baseline is used throughout the report to judge how much the models learn.

1. Data exploration (EDA)

1.1 **Import libraries:** pandas, numpy, matplotlib, seaborn and scikit-learn.

```
import re
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.feature_extraction.text import TfidfVectorizer, CountVectorizer, ENGLISH_STOP_WORDS
from sklearn.model_selection import GroupShuffleSplit, GroupKFold, cross_val_score, GridSearchCV
from sklearn.pipeline import Pipeline
from sklearn.naive_bayes import MultinomialNB
from sklearn.linear_model import LogisticRegression
from sklearn.svm import LinearSVC
from sklearn.metrics import (accuracy_score, f1_score, classification_report,
                             confusion_matrix, ConfusionMatrixDisplay)

pd.set_option('display.max_colwidth', 120)
```

[1] ✓ 2.6s — Python

Generate | Code | Markdown

This section imports the main libraries for the analysis:

- **pandas** and **NumPy** for data manipulation.
- **matplotlib** and **seaborn** for visualization.
- **scikit-learn** for text processing and machine learning. Here CountVectorizer is used to count words; more scikit-learn tools are imported in later sections.

1.2 Loading the Data set

I loaded the dataset with pandas and used .head() to display the first five rows. Each row is one paragraph written by one model. It contains:

- the passage itself (text) and its label (model);
- the prompt it answered (prompt_id);
- the subject area and task type (domain, task_type, topic);
- its length (n_words) and the length that was requested (target_words).

```
df.head()
```

[3] ✓ 0.0s — Python

	id	prompt_id	model	text	n_words	target_words	domain	task_type	topic
0	a49059929405	g0000	claude-fable-5	Many people believe that a car's rubber tyres protect them from lightning, but the real reason is the metal body of ...	105	100	physics	explain concept	Faraday cage and lightning in cars
1	799d3aa186cb	g0000	claude-opus-4-6	Many people believe that during a lightning storm, the rubber tyres on a car act as insulators that prevent the elec...	225	211	physics	explain concept	Faraday cage and lightning in cars
2	6549ebfb5b8a	g0000	claude-opus-4-8	It's a common belief that the rubber tyres on your car protect you during a lightning storm by insulating you from t...	163	158	physics	explain concept	Faraday cage and lightning in cars
3	762eb7c6019a	g0000	claude-opus-5	You're safe in a car because its metal shell acts as a Faraday cage, conducting lightning's charge around the outsid...	44	43	physics	explain concept	Faraday cage and lightning in cars
4	f10957b7f9d9	g0000	claude-sonnet-5	When lightning strikes a car, you're protected by the metal body acting as a Faraday cage, not by the rubber tyres a...	114	109	physics	explain concept	Faraday cage and lightning in cars

1.3 Displaying Info

.info() shows that the dataset has **4,998 rows and 9 columns**. Seven columns are text (id, prompt_id, model, text, domain, task_type, topic) and two are integers (n_words, target_words). Every column has 4,998 non-null values, so there are **no missing values**.

```
df.info()
```

[4] ✓ 0.0s Python

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 4998 entries, 0 to 4997
Data columns (total 9 columns):
 #   Column        Non-Null Count  Dtype 
---  ------        --------------  ----- 
 0   id            4998 non-null   object
 1   prompt_id     4998 non-null   object
 2   model         4998 non-null   object
 3   text          4998 non-null   object
 4   n_words       4998 non-null   int64 
 5   target_words  4998 non-null   int64 
 6   domain        4998 non-null   object
 7   task_type     4998 non-null   object
 8   topic         4998 non-null   object
dtypes: int64(2), object(7)
memory usage: 351.6+ KB
```

1.4 Class distribution

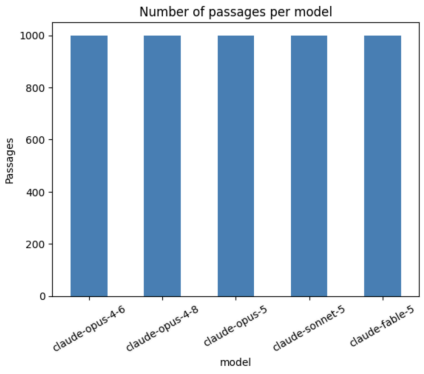
The classes are **almost perfectly balanced**. Each model answered the same 1,000 prompts, and only two claude-fable-5 responses were removed because the model refused to answer. Unlike the wine dataset, no class weighting is needed, and accuracy is a fair metric here. With five balanced classes, a random guess would be correct **20%** of the time; this is the baseline for the models.

```
print(df['model'].value_counts())

df['model'].value_counts().plot(kind='bar', color='steelblue')
plt.title('Number of passages per model')
plt.ylabel('Passages')
plt.xticks(rotation=30)
plt.show()
```

[5] ✓ 0.1s

```
model
claude-opus-4-6    1000
claude-opus-4-8    1000
claude-opus-5      1000
claude-sonnet-5    1000
claude-fable-5      998
Name: count, dtype: int64
```



1.5 Text length per model

model	n_words	avg_word_len	length_ratio
claude-fable-5	117.02	6.55	1.01
claude-opus-4-6	122.70	6.52	1.07
claude-opus-4-8	116.88	6.46	1.03
claude-opus-5	118.61	6.29	1.03
claude-sonnet-5	123.79	6.61	1.09

Passages range from **34 to 297 words** (median 106). The distribution is right-skewed, because the length requested in the prompts varied between 36 and 274 words.

The models write texts of **very similar length**. Claude-sonnet-5 and claude-opus-4-6 write slightly longer passages (about 123 words) and claude-opus-4-8 the shortest (about 117), but the difference is only around 6 words, and the boxplots overlap almost completely. The reason is that all models were asked for the same length and followed the instruction closely. **Text length alone cannot identify the model**, so the classifier will have to rely on word choice and style.

1.6 Most common words

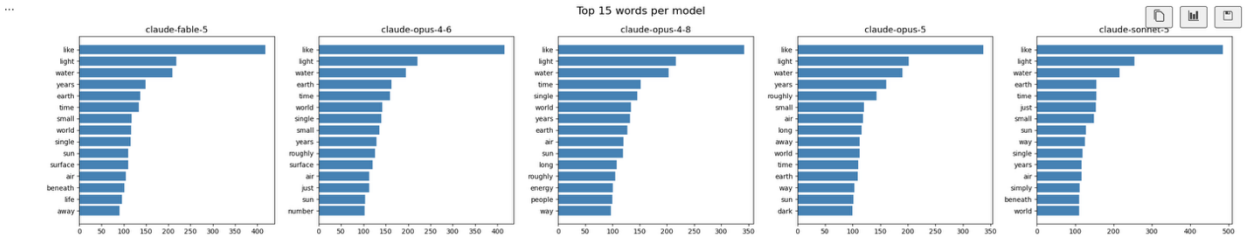
Stop words (*the, and, of, is* ...) were removed, because they are the most frequent words in any English text and would hide everything else.

Overall, the most common words are *like* (1,998), *light* (1,112), *water* (1,014), *time*, *earth*, *years*, *world*, *small*, *single* and *air*. They mostly reflect the **topics** of the prompts, as physics is the largest domain. The frequent use of *like*, *roughly*, *just* and *simply* also shows that the models often explain ideas through comparisons and approximations.

1.7 Most frequent words per model

To visualize vocabulary of each model, I plotted the 15 most frequent words per model (stop words removed). The five charts look very similar and are dominated by topic words such as *like*, *light*, *water* and *earth*. This confirms the observation from section 1.6: the models write about the same topics with similar vocabulary. Only small differences appear, for example *roughly* for claude-opus-5 and claude-opus-4-6, and *just* for claude-sonnet-5. This means the

differences between the models are subtle and need to be found with statistical methods such as TF-IDF.



1.8 Example passages

To compare the styles directly, I printed the answers of all five models to the same prompt, about why people are safe from lightning inside a car:

- **claude-fable-5 (105 words):** *"Many people believe that a car's rubber tyres protect them from lightning, but the real reason is the metal body of the vehicle itself…"*
- **claude-opus-4-6 (225 words):** *"Many people believe that during a lightning storm, the rubber tyres on a car act as insulators… While this is an understandable assumption, it is actually a misconception."*
- **claude-opus-4-8 (163 words):** *"It's a common belief that the rubber tyres on **your** car protect **you** during a lightning storm… travelled through kilometres of air*—**an excellent insulator**—*to reach the ground."*
- **claude-opus-5 (44 words):** *"**You're** safe in a car because its metal shell acts as a Faraday cage…"*

All models give the same correct explanation (the Faraday cage), but their styles differ:

- claude-opus-4-6 is the most formal and elaborate (*"While this is an understandable assumption"*);
- claude-opus-4-8 speaks directly to the reader (*your*, *you*) and uses dashes;
- claude-opus-5 is short and direct.

These small habits in connector words, formality and addressing the reader are exactly what the classifier will try to learn.

```
example_id = df['prompt_id'].iloc[0]
for _, row in df[df['prompt_id'] == example_id].iterrows():
    print(f"[{row['model']}] ({row['n_words']} words)\n{row['text'][:400]}...\n")
```

[60] ✓ 0.0s

2. Text Preprocessing

2.1 Checking for missing and duplicate passages

Before cleaning the text, I checked for missing values and duplicate passages. All columns have **0 missing values**, and there are **0 duplicate passages** and **0 duplicate ids**. The dataset was already cleaned by its author: the only problematic responses (two refusals by claude-fable-5) were removed before publication. No rows need to be removed, and the dataset keeps all **4,998 passages**.

2.2 Cleaning the text

I applied four cleaning steps to every passage:

1. **Lower case:** *"Lightning"* and *"lightning"* should count as the same word.
2. **Remove URLs:** links carry no style information. No passage contains a URL, but the step makes the code safe for new data.
3. **Remove numbers:** 1,645 passages (33%) contain numbers such as years or measurements. Numbers depend on the topic, not on the model's style, and would only add noise.
4. **Remove extra spaces:** to keep the text tidy.

```
[63]
import re

def clean_text(text):
    text = text.lower()                              # 1. lower case
    text = re.sub(r'http\S+|www\.\S+', ' ', text)    # 2. remove URLs
    text = re.sub(r'\d+', ' ', text)                 # 3. remove numbers
    text = re.sub(r'\s+', ' ', text).strip()         # 4. remove extra spaces
    return text

df['clean_text'] = df['text'].apply(clean_text)

print('Passages with URLs:', df['text'].str.contains(r'http|www\.').sum())
print('Passages with numbers:', df['text'].str.contains(r'\d').sum())
```

✓ 0.2s

```
Passages with URLs: 0
Passages with numbers: 1645
```

Punctuation: keep or remove? Usually, punctuation is removed in text mining. In this project, however, punctuation can be part of a model's **writing style**, so I counted some punctuation marks per model:

```
'—' {'claude-fable-5': 1249, 'claude-opus-4-6': 1174, 'claude-opus-4-8': 911, 'claude-opus-5': 1304, 'claude-sonnet-5': 1246}
';' {'claude-fable-5': 314, 'claude-opus-4-6': 163, 'claude-opus-4-8': 252, 'claude-opus-5': 542, 'claude-sonnet-5': 251}
':' {'claude-fable-5': 745, 'claude-opus-4-6': 385, 'claude-opus-4-8': 492, 'claude-opus-5': 806, 'claude-sonnet-5': 589}
'(' {'claude-fable-5': 85, 'claude-opus-4-6': 110, 'claude-opus-4-8': 69, 'claude-opus-5': 69, 'claude-sonnet-5': 117}
'?' {'claude-fable-5': 62, 'claude-opus-4-6': 45, 'claude-opus-4-8': 66, 'claude-opus-5': 41, 'claude-sonnet-5': 63}
'!' {'claude-fable-5': 25, 'claude-opus-4-6': 19, 'claude-opus-4-8': 23, 'claude-opus-5': 6, 'claude-sonnet-5': 11}
```

The differences are clear:

- **claude-opus-5** uses more than three times as many semicolons as claude-opus-4-6 (542 vs 163), and the most colons.
- **claude-opus-4-8** uses the fewest dashes.
- **claude-sonnet-5** uses brackets the most.

This is the **trade-off**. Removing punctuation makes the text simpler and the vocabulary smaller, but it also deletes real style information. I therefore **did not remove punctuation** from the text. The TF-IDF vectorizer used in section 3 only builds features from words and ignores punctuation, so this information is not used by the model. That is a limitation I return to in the conclusion: adding punctuation counts as extra features could improve the results.

2.3 Stop words

Stop words are very common words such as *the, and, of, is, this* and *so*. scikit-learn's English list contains 318 of them. In most text mining tasks, such as classifying news by topic, they are removed because they appear in every text and say nothing about the topic. In this dataset they make up **43.7% of all words** (598,245 words before, 336,981 after removal).

In this project, however, removing them is not necessarily good. The topic is the same for all five models, because they answered the same prompts. What differs is **how** they write: connector words such as *so, thus, however, since* and *just* are exactly the kind of habits that can reveal a model, and many of them are stop words.

I therefore kept **two versions** of the text:

- no_stop_text (without stop words), used for word-frequency exploration (sections 1.6–1.7);
- clean_text (with stop words), used for the models.

In section 4.5, hyperparameter tuning tests both options (stop_words=None vs 'english') so that the data decides which works better.

```
[66]  ✓ 0.3s
```

```python
from sklearn.feature_extraction.text import ENGLISH_STOP_WORDS

def remove_stop_words(text):
    words = re.findall(r"[a-z']+", text)
    return ' '.join(w for w in words if w not in ENGLISH_STOP_WORDS)

df['no_stop_text'] = df['clean_text'].apply(remove_stop_words)

total = df['clean_text'].str.split().str.len().sum()
after = df['no_stop_text'].str.split().str.len().sum()
print(f'Words before: {total:,}  after: {after:,}  removed: {1 - after/total:.1%}')
```

```
Words before: 598,245  after: 336,981  removed: 43.7%
```

2.4 Lemmatization

```
import nltk
nltk.download('wordnet', quiet=True)
from nltk.stem import WordNetLemmatizer

lemmatizer = WordNetLemmatizer()
for word in ['delving', 'conducting', 'strikes', 'acts', 'tyres', 'passengers']:
    print(word, '->', lemmatizer.lemmatize(word, pos='v'), '/', lemmatizer.lemmatize(word))
```

[78] ✓ 0.0s

```
delving -> delve / delving
conducting -> conduct / conducting
strikes -> strike / strike
acts -> act / act
tyres -> tyres / tyre
passengers -> passengers / passenger
```

Lemmatization reduces each word to its base form (its *lemma*): *delving* → *delve*, *strikes* → *strike*, *passengers* → *passenger*. This makes the vocabulary smaller, because different forms of the same word are counted together.

I **did not apply** lemmatization in the final pipeline, for two reasons:

1. **It can remove style information.** Whether a model prefers *"it delves"* or *"delving into"* is part of its style, and lemmatization would merge them.
2. **TF-IDF with two-word phrases** (section 3) already captures much of the meaning, and the vocabulary is not too large for the models.

The code above only demonstrates how lemmatization works. It needs the extra nltk package; if it doesn't install, you can skip this subsection or keep only the explanation.

2.5 Comparing the text before and after cleaning

```
i = 0
print('ORIGINAL:\n', df['text'].iloc[i][:250], '\n')
print('CLEANED (used for the models):\n', df['clean_text'].iloc[i][:250], '\n')
print('WITHOUT STOP WORDS (used for exploration):\n', df['no_stop_text'].iloc[i][:250])
```

[79] ✓ 0.0s Python

```
ORIGINAL:
 Many people believe that a car's rubber tyres protect them from lightning, but the real reason is the metal body of the vehicle itself. When light

CLEANED (used for the models):
 many people believe that a car's rubber tyres protect them from lightning, but the real reason is the metal body of the vehicle itself. when light

WITHOUT STOP WORDS (used for exploration):
 people believe car's rubber tyres protect lightning real reason metal body vehicle lightning strikes car metal shell acts faraday cage conducting
```

The **cleaned** version is still easy to read: only the capital letters changed, because this passage contains no numbers or URLs. The version **without stop words** keeps the main content words (*rubber, tyres, lightning, metal, faraday cage*) but loses the connecting words (*that, but, the,*

when, itself), so it reads like a list of keywords. It shows the topic well, but much of the writing style is lost. This is why the cleaned version with stop words is used for classification.

3. Feature Extraction

3.1 What is TF-IDF?

Machine learning models cannot read text; they need numbers. **TF-IDF (Term Frequency – Inverse Document Frequency)** converts each passage into a list of numbers, with one number for every word (or phrase) in the vocabulary. Each number shows how important that word is for that passage.

It combines two parts:

- **TF (term frequency):** how often a word appears in the passage. A word used five times is more important to that passage than a word used once.
- **IDF (inverse document frequency):** how rare the word is across all passages. It is calculated as IDF = log(N / number of passages containing the word), where N is the total number of passages. A word found in almost every passage, such as *the*, gets a low IDF; a word found in only a few passages, such as *faraday*, gets a high IDF.

The final score is **TF × IDF**. A word scores high when it is **frequent in this passage but rare overall**, which is exactly the kind of word that can distinguish one text from another. Each passage becomes a long vector of scores, and most of its values are 0 because each passage uses only a small part of the whole vocabulary.

3.2 Splitting into training and test sets

```
from sklearn.model_selection import GroupShuffleSplit

X = df['clean_text']
y = df['model']
groups = df['prompt_id']

gss = GroupShuffleSplit(n_splits=1, test_size=0.2, random_state=42)
train_idx, test_idx = next(gss.split(X, y, groups))

X_train, X_test = X.iloc[train_idx], X.iloc[test_idx]
y_train, y_test = y.iloc[train_idx], y.iloc[test_idx]
groups_train = groups.iloc[train_idx]

print('Train passages:', len(X_train), ' Test passages:', len(X_test))
print('Train prompts:', groups_train.nunique(), ' Test prompts:', groups.iloc[test_idx].nunique())
print('Shared prompts:', len(set(groups_train) & set(groups.iloc[test_idx])))
print('\nTrain classes:', y_train.value_counts().to_dict())
print('Test classes: ', y_test.value_counts().to_dict())
```

✓ 0.3s Pyt

I split the data into **80% training and 20% testing**, and I did this **before** TF-IDF for two reasons.

1. Grouping by prompt. Each of the 1,000 prompts was answered by all five models. With a normal random split, the answers of four models to a prompt could be in the training set and the fifth in the test set. The model could then recognise the **topic** (for example "lightning and cars") instead of the **writing style**. To prevent this, I used GroupShuffleSplit with prompt_id as the group, so all five answers to a prompt always go to the same set.

2. Avoiding data leakage. TF-IDF learns from the data: which words exist, and how rare they are (IDF). If it were fitted on all passages, information from the test set would leak into training. Splitting first and fitting TF-IDF only on the training set keeps the test set completely unseen.

```
Train passages: 4000  Test passages: 998
Train prompts: 800  Test prompts: 200
Shared prompts: 0

Train classes: {'claude-fable-5': 800, 'claude-opus-4-6': 800, 'claude-opus-4-8': 800, 'claude-opus-5': 800, 'claude-sonnet-5': 800}
Test classes:  {'claude-opus-4-6': 200, 'claude-opus-4-8': 200, 'claude-opus-5': 200, 'claude-sonnet-5': 200, 'claude-fable-5': 198}
```

There are **0 shared prompts** between the two sets. Because every prompt contains one passage from each model, the grouped split also keeps the classes perfectly balanced, so stratification is not needed.

3.3 Creating TF-IDF features

I chose these settings for the vectorizer:

- **ngram_range=(1, 2):** uses single words (*metal*) and two-word phrases (*the metal*, *rubber tyres*). Phrases capture style habits that single words miss, such as *"so the"* or *"known as"*.
- **min_df=2:** removes terms that appear in only one passage, such as typos or very rare names. They cannot help classify other passages.
- **max_df=0.9:** removes terms that appear in more than 90% of passages. In this dataset that removes *the*, *and* and *a*, which appear in almost every text and carry no information. Other stop words, such as *so*, *this* and *thus*, are kept on purpose, as explained in section 2.3.
- **max_features=20000:** without a limit, the vocabulary would have 77,316 terms. Keeping the 20,000 most frequent ones reduces noise and speeds up training.
- **sublinear_tf=True:** uses 1 + log(count) instead of the raw count, so a word repeated ten times does not get ten times more weight than a word used once.

The vectorizer is **fitted on the training set only** (fit_transform); the test set is only **transformed** with the same vocabulary and IDF values (transform).

```
[81]
from sklearn.feature_extraction.text import TfidfVectorizer

tfidf = TfidfVectorizer(ngram_range=(1, 2),   # single words and two-word phrases
                        min_df=2,             # ignore terms found in fewer than 2 passages
                        max_df=0.9,           # ignore terms found in more than 90% of passages
                        max_features=20000,   # keep the 20,000 most frequent terms
                        sublinear_tf=True)    # use 1 + log(TF) instead of raw counts

X_train_tfidf = tfidf.fit_transform(X_train)  # learn vocabulary and IDF from training data only
X_test_tfidf = tfidf.transform(X_test)        # apply the same vocabulary to the test data
✓ 1.2s
```

3.4 The resulting feature matrix

```
[82]
print('Train matrix:', X_train_tfidf.shape)
print('Test matrix: ', X_test_tfidf.shape)

feature_names = tfidf.get_feature_names_out()
print('Two-word phrases:', sum(' ' in f for f in feature_names))
print('Average non-zero values per passage:', round(X_train_tfidf.nnz / X_train_tfidf.shape[0], 1))
print('Example features:', feature_names[1000:1015])

# Highest TF-IDF terms of the first training passage
import numpy as np
row = X_train_tfidf[0].toarray()[0]
top = np.argsort(row)[::-1][:10]
print('\nPassage:', X_train.iloc[0][:150], '...')
print([(feature_names[i], round(row[i], 3)) for i in top])
✓ 0.0s  Python
```

```
Train matrix: (4000, 20000)
Test matrix:  (998, 20000)
Two-word phrases: 10230
Average non-zero values per passage: 112.2
Example features: ['and touch' 'and trembling' 'and trust' 'and turned' 'and twenty'
 'and two' 'and ultimately' 'and unbroken' 'and understand'
 'and understood' 'and unhurried' 'and unremarkable' 'and used'
 'and vanish' 'and vanished']
```

- **Vocabulary:** of the 20,000 features, **9,698 are single words and 10,302 are two-word phrases**. Example features: *answers, answers the, antenna, anthropic principle, antibiotics…*
- **Sparsity:** each passage contains on average only **112 of the 20,000 terms**, so **99.4% of the values in the matrix are 0**. This is normal for text data. scikit-learn stores it as a *sparse matrix*, which keeps only the non-zero values and saves memory.

4. Models

For classification I chose three models that are widely used for text data. All three work well with large, sparse TF-IDF matrices. I also use them in cross-validation and hyperparameter tuning.

```
[83]  ✓ 0.0s
```

```python
from sklearn.naive_bayes import MultinomialNB
from sklearn.linear_model import LogisticRegression
from sklearn.svm import LinearSVC
from sklearn.metrics import (accuracy_score, f1_score, classification_report,
                             confusion_matrix, ConfusionMatrixDisplay)

labels = sorted(y.unique())

def model_report(y_test, y_pred, title=''):
    print('Accuracy:', round(accuracy_score(y_test, y_pred), 3))
    print('Macro F1-score:', round(f1_score(y_test, y_pred, average='macro'), 3))
    print(classification_report(y_test, y_pred, zero_division=0))
    ConfusionMatrixDisplay(confusion_matrix(y_test, y_pred, labels=labels),
                           display_labels=labels).plot(cmap='Blues', xticks_rotation=30)
    plt.title(title)
    plt.show()
```

A. Multinomial Naive Bayes. A probabilistic model based on Bayes' theorem. It calculates how likely each word is for each class and combines these probabilities, assuming the words are independent of each other. This assumption is "naive", but the model is very fast and a classic baseline for text classification.

B. Logistic Regression. A linear model that learns one weight (coefficient) per word for each class. A positive weight pushes a passage towards that class, a negative weight away from it. It works very well on TF-IDF data, and its coefficients show **which words matter for each model**, which I use in section 5 to answer the research question.

C. Linear Support Vector Machine (LinearSVC). It finds the boundaries (hyperplanes) that separate the classes with the largest possible margin. With thousands of TF-IDF features, the classes are often separable by straight boundaries, so the linear version is fast and usually one of the most accurate text classifiers.

Evaluation metrics: confusion matrix, precision, recall, F1-score, accuracy and macro F1-score. Because the five classes are balanced, accuracy is a fair metric here. A random guess would be correct **20%** of the time, which is the **baseline** each model must beat.

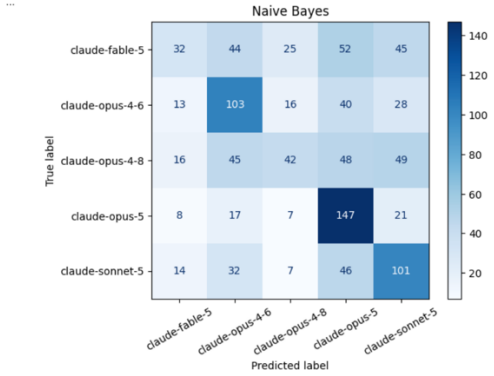
4.1 Training and evaluating Multinomial Naive Bayes

I trained this model on the training set and then used the performance metrics to evaluate it on the test set.

```
Accuracy: 0.426
Macro F1-score: 0.397
                  precision    recall  f1-score   support

 claude-fable-5       0.39      0.16      0.23       198
claude-opus-4-6       0.43      0.52      0.47       200
claude-opus-4-8       0.43      0.21      0.28       200
  claude-opus-5       0.44      0.73      0.55       200
claude-sonnet-5       0.41      0.51      0.45       200

       accuracy                           0.43       998
      macro avg       0.42      0.43      0.40       998
   weighted avg       0.42      0.43      0.40       998
```



4.1.1 Model performance analysis

A. Confusion matrix:

- **fable-5:** 32 correct predictions; 44 misclassified as opus-4-6, 25 as opus-4-8, 52 as opus-5 and 45 as sonnet-5.
- **opus-4-6:** 103 correct predictions; 13 misclassified as fable-5, 16 as opus-4-8, 40 as opus-5 and 28 as sonnet-5.
- **opus-4-8:** 42 correct predictions; 16 misclassified as fable-5, 45 as opus-4-6, 48 as opus-5 and 49 as sonnet-5.
- **opus-5:** 147 correct predictions; 8 misclassified as fable-5, 17 as opus-4-6, 7 as opus-4-8 and 21 as sonnet-5.
- **sonnet-5:** 101 correct predictions; 14 misclassified as fable-5, 32 as opus-4-6, 7 as opus-4-8 and 46 as opus-5.

B. Classification report:

- **Precision** shows the proportion of correct predictions among all the predictions made for a class.

    - fable-5: 0.39, so 39% of the passages predicted as fable-5 are correct.
    - opus-4-6: 0.43, so 43% are correct.
    - opus-4-8: 0.43, so 43% are correct.
    - opus-5: 0.44, so 44% are correct.
    - sonnet-5: 0.41, so 41% are correct.

  Precision is similar for all classes: when Naive Bayes predicts a model, it is right about 4 times out of 10.

- **Recall** measures how many passages of a class were correctly identified out of all the passages of that class.
    - fable-5: 0.16, so only 16% of fable-5 passages were found, the worst result.
    - opus-4-6: 0.52, so 52% were found.
    - opus-4-8: 0.21, so only 21% were found.
    - opus-5: 0.73, so 73% were found, the best result.
    - sonnet-5: 0.51, so 51% were found.
- **F1-score** is the harmonic mean of precision and recall, used to balance the two.
    - fable-5: 0.23, which is poor.
    - opus-4-6: 0.47, which is moderate.
    - opus-4-8: 0.28, which is poor.
    - opus-5: 0.55, the best of the five classes.
    - sonnet-5: 0.45, which is moderate.

The model predicted opus-5 **333 times**, although the test set contains only 200 opus-5 passages, while it predicted fable-5 only **83 times**. Naive Bayes is therefore **biased towards opus-5**: many passages from every other model are labelled as opus-5. That is why opus-5 has high recall but low precision.

C. Accuracy:

The accuracy is **42.6%** and the macro F1-score is **0.397**. This is more than twice the random baseline of 20%, so the model has learned real differences between the writing styles.

To summarize:

Naive Bayes clearly beats random guessing and recognizes opus-5, opus-4-6 and sonnet-5 reasonably well. However, it relies too heavily on opus-5 and recognizes fable-5 and opus-4-8 poorly. This is a known weakness of Naive Bayes: it treats every word independently, so it cannot capture combinations of words that make up a writing style.

4.2 Training and evaluating Logistic Regression

I trained this model on the training set and then used the performance metrics to evaluate it on the test set. `C=10` sets weak regularization, which lets the model use many small style signals. `max_iter=2000` gives the solver enough iterations to finish training.

4.2.1 Model performance analysis

A. Confusion matrix:

- **fable-5:** 59 correct predictions; 31 misclassified as opus-4-6, 40 as opus-4-8, 31 as opus-5 and 37 as sonnet-5.
- **opus-4-6:** 97 correct predictions; 31 misclassified as fable-5, 27 as opus-4-8, 18 as opus-5 and 27 as sonnet-5.
- **opus-4-8:** 67 correct predictions; 32 misclassified as fable-5, 39 as opus-4-6, 18 as opus-5 and 44 as sonnet-5.
- **opus-5:** 119 correct predictions; 25 misclassified as fable-5, 18 as opus-4-6, 16 as opus-4-8 and 22 as sonnet-5.
- **sonnet-5:** 106 correct predictions; 29 misclassified as fable-5, 18 as opus-4-6, 23 as opus-4-8 and 24 as opus-5.

B. Classification report:

- **Precision** shows the proportion of correct predictions among all the predictions made for a class.
  - fable-5: 0.34, so 34% of the passages predicted as fable-5 are correct, the lowest.
  - opus-4-6: 0.48, so 48% are correct.
  - opus-4-8: 0.39, so 39% are correct.
  - opus-5: 0.57, so 57% are correct, the most reliable prediction.
  - sonnet-5: 0.45, so 45% are correct.
- **Recall** measures how many passages of a class were correctly identified out of all the passages of that class.
  - fable-5: 0.30, so 30% were found (only 16% with Naive Bayes).
  - opus-4-6: 0.48, so 48% were found.
  - opus-4-8: 0.34, so 34% were found (21% with Naive Bayes).
  - opus-5: 0.59, so 59% were found, the best result.
  - sonnet-5: 0.53, so 53% were found.
- **F1-score** is the harmonic mean of precision and recall, used to balance the two.
  - fable-5: 0.32, which is poor.
  - opus-4-6: 0.48, which is moderate.
  - opus-4-8: 0.36, which is poor to moderate.
  - opus-5: 0.58, the best of the five classes.
  - sonnet-5: 0.49, which is moderate.

The predictions are now **spread much more evenly** across the classes: the model predicted fable-5 176 times, opus-4-6 203, opus-4-8 173, opus-5 210 and sonnet-5 236, each close to the true 200. Unlike Naive Bayes, which predicted opus-5 333 times, Logistic Regression does not favor one model. sonnet-5 is predicted slightly too often, mainly because opus-4-8 passages are mistaken for it (44 times).

C. Accuracy:

The accuracy is **44.9%** and the macro F1-score is **0.445**. Both are higher than Naive Bayes (42.6% and 0.397) and more than twice the random baseline of 20%.

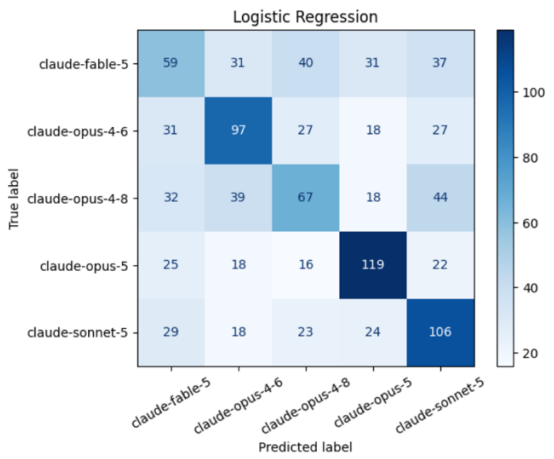
To summarize:

Logistic Regression is more accurate and much more balanced than Naive Bayes. The biggest gains are for the two hardest classes: recall for fable-5 almost doubled (0.16 → 0.30) and opus-4-8 improved from 0.21 to 0.34. opus-5 and sonnet-5 remain the easiest models to recognize, while fable-5 and opus-4-8 are still the hardest. Their passages are confused with all the other models, which suggests their writing style is the least distinctive. Because Logistic Regression learns a weight for each word, it can combine many small style signals, which is why it performs better than Naive Bayes.

```
Accuracy: 0.449
Macro F1-score: 0.445
                  precision    recall  f1-score   support

 claude-fable-5       0.34      0.30      0.32       198
claude-opus-4-6       0.48      0.48      0.48       200
claude-opus-4-8       0.39      0.34      0.36       200
  claude-opus-5       0.57      0.59      0.58       200
claude-sonnet-5       0.45      0.53      0.49       200

       accuracy                           0.45       998
      macro avg       0.44      0.45      0.44       998
   weighted avg       0.44      0.45      0.44       998
```



4.3 Training and evaluating the Linear SVM

I trained this model on the training set and then used the performance metrics to evaluate it on the test set. C controls the margin: a smaller C gives a wider margin and a simpler model, which reduces overfitting to the many rare words in the TF-IDF matrix. I used C=0.5.

4.3.1 Model performance analysis

A. Confusion matrix:

- **fable-5:** 50 correct predictions; 35 misclassified as opus-4-6, 33 as opus-4-8, 43 as opus-5 and 37 as sonnet-5.
- **opus-4-6:** 102 correct predictions; 26 misclassified as fable-5, 22 as opus-4-8, 23 as opus-5 and 27 as sonnet-5.
- **opus-4-8:** 57 correct predictions; 27 misclassified as fable-5, 44 as opus-4-6, 28 as opus-5 and 44 as sonnet-5.
- **opus-5:** 134 correct predictions; 19 misclassified as fable-5, 18 as opus-4-6, 11 as opus-4-8 and 18 as sonnet-5.
- **sonnet-5:** 110 correct predictions; 26 misclassified as fable-5, 19 as opus-4-6, 19 as opus-4-8 and 26 as opus-5.

B. Classification report:

- **Precision** shows the proportion of correct predictions among all the predictions made for a class.
  - fable-5: 0.34, so 34% of the passages predicted as fable-5 are correct, the lowest.
  - opus-4-6: 0.47, so 47% are correct.
  - opus-4-8: 0.40, so 40% are correct.
  - opus-5: 0.53, so 53% are correct, the highest.
  - sonnet-5: 0.47, so 47% are correct.
- **Recall** measures how many passages of a class were correctly identified out of all the passages of that class.
  - fable-5: 0.25, so only 25% were found.
  - opus-4-6: 0.51, so 51% were found.
  - opus-4-8: 0.28, so only 28% were found.
  - opus-5: 0.67, so 67% were found, the best result.
  - sonnet-5: 0.55, so 55% were found.
- **F1-score** is the harmonic mean of precision and recall, used to balance the two.
  - fable-5: 0.29, which is poor.
  - opus-4-6: 0.49, which is moderate.
  - opus-4-8: 0.33, which is poor.
  - opus-5: 0.59, the best of the five classes.
  - sonnet-5: 0.50, which is moderate.

The model predicted opus-5 **254 times** and sonnet-5 **236 times**, but fable-5 only **148 times** and opus-4-8 only **142 times** (each class has about 200 passages). The SVM therefore **favours the easier classes**: it finds more opus-5, opus-4-6 and sonnet-5 passages than Logistic Regression, but fewer fable-5 and opus-4-8 passages.

C. Accuracy:

The accuracy is **45.4%** and the macro F1-score is **0.441**. The SVM has the **highest accuracy of the three models**, but its macro F1-score is slightly lower than Logistic Regression's (0.445). It is still more than twice the random baseline of 20%.

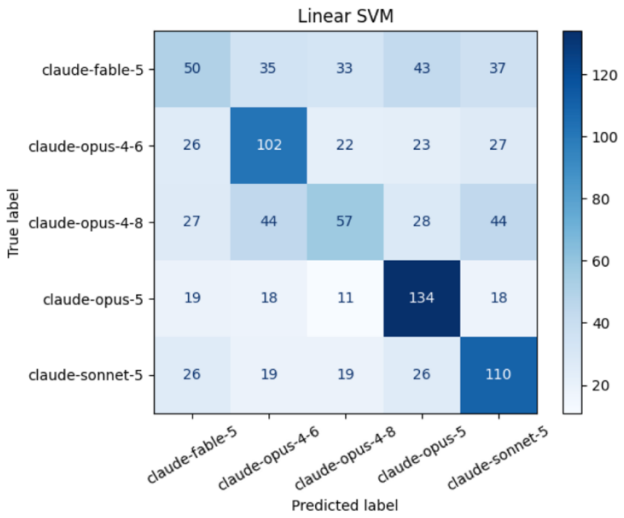
To summarize:

The Linear SVM and Logistic Regression perform almost equally; the differences are less than 1%. The SVM gets slightly more passages right in total, mainly because it recognizes opus-5 (67%) and sonnet-5 (55%) better. However, it is less balanced: it predicts the difficult classes less often, so fable-5 recall drops to 0.25 (0.30 with Logistic Regression) and opus-4-8 to 0.28 (0.34 with Logistic Regression). As with the other models, **opus-5 is the easiest model to recognize, and fable-5 and opus-4-8 are the hardest**.

```
Accuracy: 0.454
Macro F1-score: 0.441
                  precision    recall  f1-score   support

  claude-fable-5       0.34      0.25      0.29       198
 claude-opus-4-6       0.47      0.51      0.49       200
 claude-opus-4-8       0.40      0.28      0.33       200
   claude-opus-5       0.53      0.67      0.59       200
 claude-sonnet-5       0.47      0.55      0.50       200

        accuracy                           0.45       998
       macro avg       0.44      0.45      0.44       998
    weighted avg       0.44      0.45      0.44       998
```



4.4 Cross-validation score

Cross-validation gives a more reliable estimate of performance than a single train/test split. The training data is divided into 5 parts (folds). Each model is trained on 4 folds and tested on the remaining one. This is repeated 5 times, so every fold is used once for testing, and the average score is reported.

I made two choices to keep the evaluation fair:

- **GroupKFold with prompt_id:** all five passages answering the same prompt are always in the same fold. The models are always tested on **prompts they have never seen**, so they cannot use topic words to recognize a model.
- **TF-IDF inside a Pipeline:** the vectorizer is re-fitted on the training folds every time, so it never sees the validation fold. This prevents data leakage.

```
[112] ✓ 12.7s
```

```
from sklearn.model_selection import GroupKFold, cross_val_score
from sklearn.pipeline import Pipeline

gkf = GroupKFold(n_splits=5)

def make_tfidf():
    return TfidfVectorizer(ngram_range=(1, 2), min_df=2, max_df=0.9, sublinear_tf=True)

pipelines = {
    'Naive Bayes': Pipeline([('tfidf', make_tfidf()), ('clf', MultinomialNB())]),
    'Logistic Regression': Pipeline([('tfidf', make_tfidf()), ('clf', LogisticRegression(max_iter=2000, C=10))]),
    'Linear SVM': Pipeline([('tfidf', make_tfidf()), ('clf', LinearSVC(C=0.5))]),
}

print('Cross-validation accuracy (5 folds, grouped by prompt)')
for name, pipe in pipelines.items():
    scores = cross_val_score(pipe, X_train, y_train, groups=groups_train, cv=gkf, scoring='accuracy')
    print(f'{name}: {scores.round(3)}  Avg: {scores.mean():.3f}')
```

```
Cross-validation accuracy (5 folds, grouped by prompt)
Naive Bayes: [0.382 0.382 0.381 0.361 0.382]  Avg: 0.378
Logistic Regression: [0.429 0.428 0.424 0.41  0.455]  Avg: 0.429
Linear SVM: [0.428 0.414 0.421 0.412 0.446]  Avg: 0.424
```

From the output, I observed that:

- **Logistic Regression has the highest average accuracy (42.9%)**, followed closely by the Linear SVM (42.4%). The difference is only 0.5%, so the two models are practically equal.
- **Naive Bayes is clearly the weakest (37.8%)**, about 5% below the other two. This confirms the test-set results from sections 4.1–4.3.
- **The results are stable.** Each model's scores vary by only about 2–4% between folds, so the performance does not depend on one lucky split. Naive Bayes is the most stable (36.1–38.2%), but at a lower level.
- **The cross-validation scores are 2–5% lower than the test-set scores** (for example 42.9% vs 44.9% for Logistic Regression). In cross-validation each model is trained on only 3,200 passages instead of 4,000, which suggests that **more training data would improve the results**.
- All models remain **about twice as good as the random baseline of 20%**.

To summarize:

Cross-validation confirms that Logistic Regression and the Linear SVM are the best models and that Naive Bayes performs worst. On the test set the SVM was slightly better (45.4% vs 44.9%), while in cross-validation Logistic Regression was slightly better (42.9% vs 42.4%). Since the order switches depending on the data, the differences between these two models are within normal variation, and neither is clearly superior.

4.5 Hyperparameter tuning

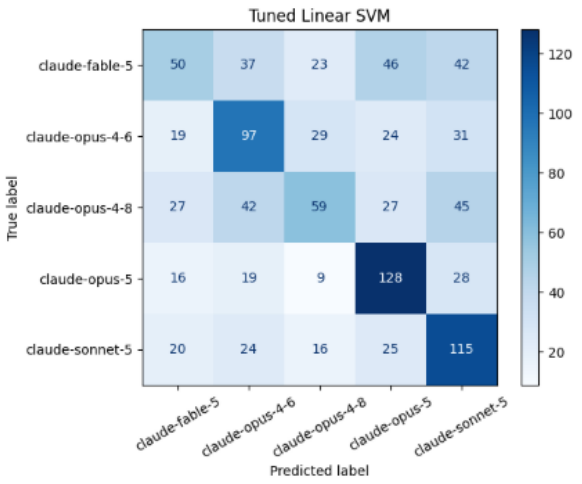
Hyperparameters are settings that control how a model learns. They are not learned from the data, so they must be chosen. I used **GridSearchCV**, which tries every combination of the values in a grid and evaluates each one with cross-validation.

I tuned the **Linear SVM**, because it had the highest test accuracy (45.4%). To keep the evaluation fair, I used:

- **GroupKFold** (5 folds, grouped by prompt), so validation prompts are never seen during training;
- the **macro F1-score** as the scoring metric, so all five models count equally;
- the **Pipeline** from section 4.4, so both the TF-IDF settings and the SVM settings could be tuned together.

The grid contains 2 × 2 × 2 × 3 = **24 combinations**, each evaluated on 5 folds:

- `tfidf__ngram_range`: single words only `(1, 1)`, or words and two-word phrases `(1, 2)`;
- `tfidf__stop_words`: keep stop words (`None`) or remove them (`'english'`). This **tests the decision from section 2.3**;
- `tfidf__sublinear_tf`: log-scaled (`True`) or raw (`False`) word counts;
- `clf__C`: the margin strength of the SVM (0.1, 0.5 or 1).



4.5.1 Best parameters

From the output, the best parameters are:

- **C = 1:** a slightly narrower margin than the C=0.5 used in section 4.3, so the model fits the training data more closely.
- **ngram_range = (1, 2):** two-word phrases help, because they capture habits such as *"so the"* or *"known as"*.
- **stop_words = None:** keeping stop words works better than removing them.
- **sublinear_tf = False:** raw word counts work slightly better than log-scaled counts.

The best cross-validation macro F1-score was **0.414**.

The most important finding is **stop_words = None**. It confirms the decision from section 2.3: in this task, small common words such as *so, thus, this* and *just* are part of each model's style. Removing them deletes useful information and makes the models worse.

4.5.2 Model performance analysis

A. Confusion matrix:

- **fable-5:** 50 correct predictions; 37 misclassified as opus-4-6, 23 as opus-4-8, 46 as opus-5 and 42 as sonnet-5.
- **opus-4-6:** 97 correct predictions; 19 misclassified as fable-5, 29 as opus-4-8, 24 as opus-5 and 31 as sonnet-5.
- **opus-4-8:** 59 correct predictions; 27 misclassified as fable-5, 42 as opus-4-6, 27 as opus-5 and 45 as sonnet-5.
- **opus-5:** 128 correct predictions; 16 misclassified as fable-5, 19 as opus-4-6, 9 as opus-4-8 and 28 as sonnet-5.
- **sonnet-5:** 115 correct predictions; 20 misclassified as fable-5, 24 as opus-4-6, 16 as opus-4-8 and 25 as opus-5.

B. Classification report:

- **Precision** ranges from 0.38 (fable-5) to 0.51 (opus-5): opus-4-6 0.44, opus-4-8 0.43 and sonnet-5 0.44.
- **Recall** stayed almost the same as before tuning (values before tuning in brackets):
  - fable-5: 0.25 (0.25)
  - opus-4-6: 0.48 (0.51)
  - opus-4-8: 0.29 (0.28)
  - opus-5: 0.64 (0.67)
  - sonnet-5: 0.57 (0.55)

- **F1-score** is best for opus-5 (0.57), followed by sonnet-5 (0.50) and opus-4-6 (0.46), and lowest for opus-4-8 (0.35) and fable-5 (0.30).

The tuned model is even more concentrated on the easier classes: it predicted sonnet-5 **261 times** and opus-5 **250 times**, but fable-5 only **132 times** and opus-4-8 only **136 times**.

C. Accuracy:

The tuned SVM reached an accuracy of **45.0%** and a macro F1-score of **0.437**, slightly lower than the untuned SVM (45.4% and 0.441).

To summarize:

Hyperparameter tuning **did not improve** the results on the test set; they are about 0.4% lower. This difference is very small: on 998 test passages it is only 4 passages, which is within normal variation. GridSearchCV chooses the settings that work best **on average across five folds**, not on this test set, so the default settings were already close to optimal.

The main value of the tuning was to **confirm which preprocessing choices matter**: keeping stop words and using two-word phrases both give better results. This supports the decisions made in sections 2 and 3.

4.6 Model Comparison

From the comparison:

- **All models are more than twice as good as random guessing** (20%).
- **Naive Bayes is clearly the weakest**, about 3–5% below the other models, and strongly biased towards opus-5.
- **Logistic Regression and the Linear SVM are practically equal.** The SVM has the highest test accuracy (45.4%), while Logistic Regression has the highest macro F1-score (0.445) and the highest cross-validation accuracy (42.9%). All differences between them are below 1%.
- **Tuning did not improve the SVM**, but it confirmed that stop words and two-word phrases are useful.

I chose Logistic Regression as the final model, for three reasons:

1. It has the **highest cross-validation accuracy**, the most reliable estimate because it is averaged over five folds.

2. It has the **highest macro F1-score** and the **most balanced predictions**. It recognizes the difficult classes better than the SVM: fable-5 recall 0.30 vs 0.25, and opus-4-8 0.34 vs 0.28.
3. Its coefficients are **easy to interpret**, so it can directly answer the research question *"Which words give away each AI model?"* in section 5.

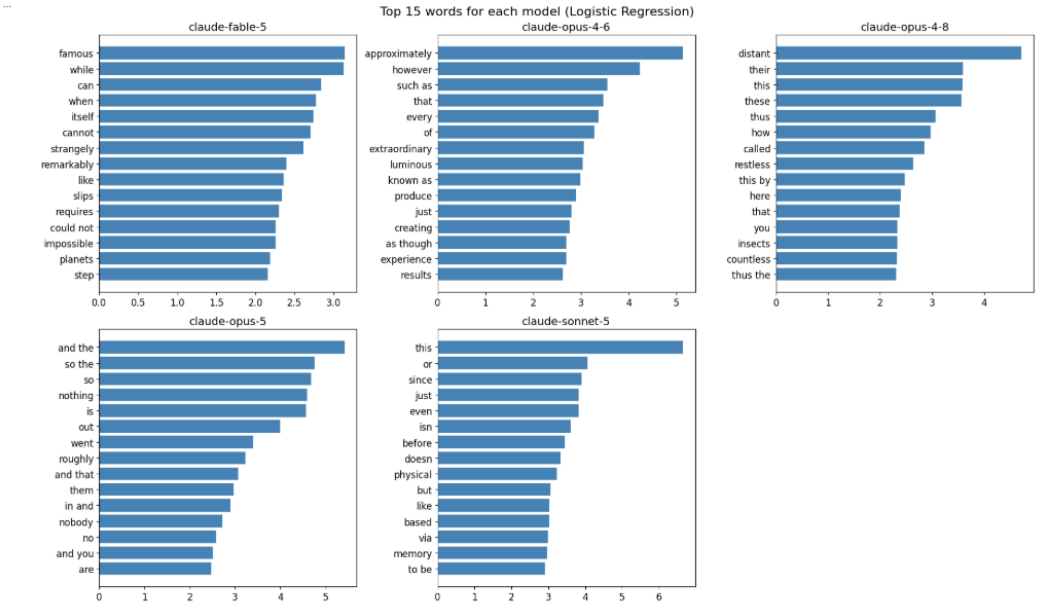
Across all models, **opus-5 is the easiest model to recognize** (recall 59–73%), followed by sonnet-5 and opus-4-6, while **fable-5 and opus-4-8 are the hardest** (recall 16–34%). Their style overlaps the most with the other models.

	Accuracy	Macro F1
Logistic Regression	0.449	0.445
Linear SVM	0.454	0.441
Linear SVM (tuned)	0.450	0.437
Naive Bayes	0.426	0.397

5. Which words give away each AI model?

5.1 Top words per model

To answer the second research question, I used the coefficients of the final model, **Logistic Regression**. It learns one weight per word (or two-word phrase) for each class: a large positive weight means the word pushes a passage towards that model. For each model I plotted the 15 terms with the highest weights.



The most typical words of each model, and the style they suggest, are:

- **claude-opus-5:** *so, so the, and the, nothing, nobody, went, roughly.* A plain, story-like style with simple connectors; it also uses the most semicolons (section 2.2).
- **claude-opus-4-6:** *approximately, however, such as, known as, extraordinary, luminous.* A formal, explanatory style with slightly elaborate adjectives.
- **claude-opus-4-8:** *thus, this, these, here, called, distant, you.* Academic connectors; it also addresses the reader directly most often.
- **claude-sonnet-5:** *since, just, even, but, doesn't, isn't.* A conversational style with contractions.
- **claude-fable-5:** *while, remarkably, strangely, itself, cannot, could not.* A literary style that likes words of surprise and negations.

The most important observation is that the typical words are **not topic words**, such as *light* or *water*, but **function words and connectors**, such as *so, thus, however, since* and *just*. This explains why keeping stop words gave better results in the hyperparameter tuning (section 4.5): the fingerprint of each model lies in the small words that connect ideas, not in what it writes about.

5.2 Which models are confused most often?

To see which models write most similarly, I removed the correct predictions from the confusion matrix of the tuned Linear SVM and listed the most frequent mistakes.

```
Most frequent mistakes (true -> predicted):
claude-fable-5   claude-opus-5      46
claude-opus-4-8  claude-sonnet-5    45
claude-fable-5   claude-sonnet-5    42
claude-opus-4-8  claude-opus-4-6    42
claude-fable-5   claude-opus-4-6    37
dtype: int64
```

All five of the most frequent mistakes involve **fable-5 or opus-4-8**, the two hardest models to recognise. Their passages are spread over all the other models, which suggests that they do not have a strong style of their own: fable-5 is most often taken for opus-5, and opus-4-8 for sonnet-5 or opus-4-6. In contrast, opus-5 is rarely confused with other models, which matches its distinctive vocabulary in section 5.1.

6. Conclusion

The aim of this project was to find out whether the model that wrote a passage can be identified from its text alone, and which words give away each model. I used 4,998 passages written by five Claude models in answer to the same 1,000 prompts. After exploring and cleaning the data, I:

- converted the text into TF-IDF features;
- split the data **by prompt**, so that the models were always tested on unseen topics;
- trained three classifiers: Multinomial Naive Bayes, Logistic Regression and a Linear SVM.

Research question 1: can the model be identified from the text alone? The answer is **yes, partly**.

- The best models identified the correct model in about **45% of the test passages** (Linear SVM 45.4%, Logistic Regression 44.9%), more than twice the random baseline of 20%.
- Naive Bayes was clearly weaker (42.6%) and biased towards opus-5.
- Cross-validation confirmed these results (42.9% for Logistic Regression).
- Hyperparameter tuning did not improve the scores, but it showed that keeping stop words and using two-word phrases works best.

I chose **Logistic Regression** as the final model because it had the best cross-validation accuracy, the highest macro F1-score (0.445) and the most balanced predictions.

Research question 2: which words give each model away? Each model has characteristic **connector words and stylistic habits**:

- opus-5 prefers *so* and simple narration;
- opus-4-6 uses formal words such as *approximately* and *however*;
- opus-4-8 uses academic connectors such as *thus*;
- sonnet-5 uses contractions and *since* or *just*;
- fable-5 uses literary words such as *remarkably* and *strangely*.

opus-5 is the easiest model to recognize, while **fable-5 and opus-4-8 are the hardest**, because their style overlaps the most with the other models.